
Middle East Technical University


Department of Computer Engineering

CENG 336

Introduction to Embedded Systems Development

Spring 2025–2026

THE3

Three-Port EV Charging Cabinet

Due: 23 May 2026, 23:55

Submission: via **ODTUClass**

1 Introduction

In this assignment you will implement the local controller of a three-port EV charging cabinet on the PIC18F8722 development board.

The cabinet receives commands from a Python site supervisor over EUSART, reads a thermal sensor through the ADC, and authorizes a cabinet-wide effective current limit that respects both the supervisor's request and a physical-safety cap derived from the live thermal reading. The supervisor independently connects or disconnects each of the three ports. The cabinet reports its state back to the supervisor periodically.

This is a group assignment. Clarifications and announcements will be posted on **ODTUClass**.

2 Overview

The system you are writing firmware for is a three-port EV charging cabinet that talks to a remote site supervisor. Figure 1 shows it in a nominal operating snapshot.

A public EV charging station has several ports for vehicles. This cabinet has three. The supervisor sets a current limit via `$LIMxx#`; the cabinet derives a thermal cap from the live ADC reading. The effective limit `ee = min(requested_limit, thermal_cap)` is cabinet-wide (applies to all connected ports, not split between them) and is reported in every status frame.

Vehicles plug into and out of the three ports independently, and the supervisor signals each connection event with a port-indexed command (`$CONp#`, `$DISp#`). The assignment does not model current allocation between ports: `ee` is cabinet-wide, and the `c` field of `STS` only reports which ports are physically connected.

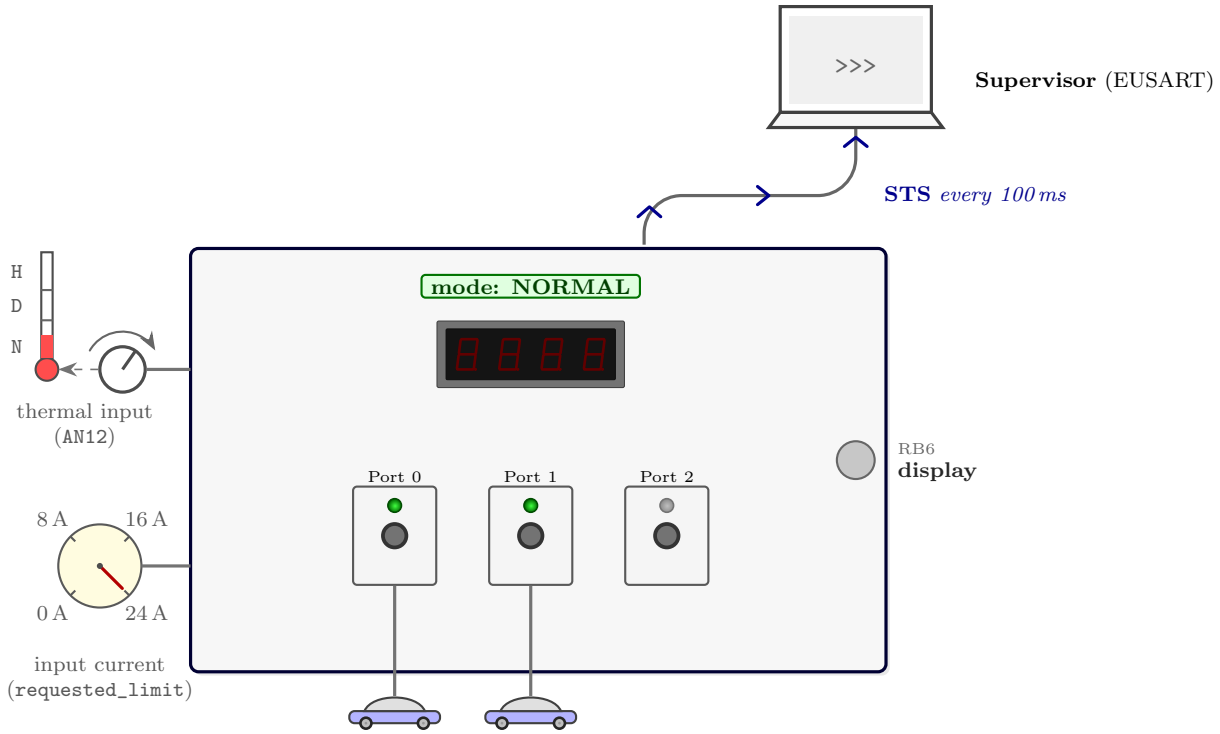

Figure 1: Three-port cabinet, nominal snapshot.

Your firmware on the PIC18F8722 *is* the cabinet's controller. The dev-board hardware plays the cabinet's local front panel:

- **Potentiometer (`AN12 / RH4`)** the thermal sensor; turn it to simulate temperature changes.
- **Pushbutton `RB6`** toggles the local 7-segment display between its two pages (Section 6.2).
- **7-segment display (`PORTJ`, `PORTH[0:3]`)** the cabinet's local readout.

Everything else (the supervisor, the cars plugging and unplugging, the limit requests) is played by a Python program on the host PC. The two parties talk over a single serial cable.

3 Hand-Out Instructions

The hand-out contains the simulator and its supporting files:

- `evCabinetSimulator.py` the Python supervisor that reads the scenario file, drives the serial connection, and displays the cabinet's reported state.
- `the3_template.c` an optional C starter file with byte-level RX/TX queues and EUSART interrupt structure. You may use it, ignore it, or rewrite it; grading is based on the behavior in this specification.
- `sample_scenario.json` a deterministic scenario for checking the simulator and cabinet interaction.

- `pragmas.h` the standard configuration pragmas for the PIC18F8722 at 40 MHz. Include it from your C source.

Running the simulator

Lab machines have all dependencies pre-installed; the same commands work on your own machine after a one-time install. The defaults are `/dev/ttyUSB0` at 115200 baud; pass `--port <DEVICE>` if your board shows up under another name (`/dev/ttyACMn`, `COMn`, etc.).

```
# install dependencies (your own machine only; lab is preinstalled)
$ pip install pyserial pygame

# play the canonical 7-second scenario from the hand-out
$ python evCabinetSimulator.py sample_scenario.json

# free-form, no scenario; manual probing via the pygame UI
$ python evCabinetSimulator.py

# override the default serial port
$ python evCabinetSimulator.py --port /dev/ttyACM0 sample_scenario.json

# text wire trace to stdout, no pygame UI
$ python evCabinetSimulator.py --log-only sample_scenario.json
```

4 Simulator

The simulator is a Python site supervisor. It can play a JSON scenario file, or run without one for manual probing through the UI. The command and message names used here are defined formally in Sections 5.1 and 6.1. The simulator cycles through the three modes shown in Figure 2.

Figure 2: Simulator modes. Above each arrow: trigger; below: emitted frame.

Mode	Simulator behavior	Cabinet behavior
`WAITING`	Initial mode after launch; no frames are sent.	Idle until `$GO#` arrives.
`ACTIVE`	Plays scheduled supervisor commands, if a scenario is loaded, and sends manual UI commands.	Runs the control loop, samples the ADC, drives the 7-segment display, and transmits one frame per 100 ms tick.
`END`	Terminal mode after `$END#`; timing statistics are finalised.	Enters its terminal silent state.

`$GO#` and `$END#` are emitted by the simulator itself. In a scenario JSON, `duration_s` sets the run length, and the `events` list contains only in-run commands such as `$CONp#`, `$DISp#`, and `$LIMxx#`.

Figure 3 shows the simulator window during a run.

Figure 3: Simulator window during a run.

(1) Cabinet bays. A car shows under bay `p` when bit `p` of `STS.c` is 1. Display-only; click controls are at **(8)**.

(2) Thermal gauge. Bar height tracks `STS.xxxx`; bar colour follows `STS.m` (green `N`, amber `D`, red `H`). To change the input, turn the dev-board potentiometer.

(3) Wire-trace terminal. Shows serial traffic in time order. In each row, `t` is seconds since `$GO#`; `->` means supervisor-to-cabinet and `<-` means cabinet-to-supervisor.

- During `ACTIVE`, the cabinet sends one frame every 100 ms: an `ACK` if a command was accepted, otherwise an `STS`.
- For readability, the simulator folds consecutive `STS` frames with the same `m`, `c`, and `ee`. $\times N$ means that row represents N wire `STS` frames. This is display-only; firmware must still transmit every 100 ms.
- $\Delta = N$ ms is the time since the previous wire `STS`; it may be larger when an `ACK` used a tick.
- Changes only in `adc` do not create a new `STS` row; the trace folds by `m`, `c`, and `ee`. Watch the thermal gauge for the live `adc` value.

(4) RUN status. Phase (`WAITING`/`ACTIVE`/`END`) and elapsed time since `$GO#`.

(5) REQ LIMIT. The most recent `$LIMxx#` the supervisor sent. `-`/`+` step through `00`/`08`/`16`/`24`.

(6) EFF LIMIT. `ee` from the most recent `STS`, i.e. `min(REQ LIMIT, thermal cap)`.

(7) GO / END buttons. Manually emit `$GO#` (only in `WAITING`) or `$END#` (only in `ACTIVE`).

(8) BAY plates. A click on plate `p` sends `$CONp#` or `$DISp#`. The plate updates only when the next `STS` confirms the bit in `c`, since `ACK01`/`ACK02` do not encode the port number.

5 Inputs to the Cabinet

This part covers the three sources of input the cabinet processes during a run: framed supervisor commands over EUSART, the thermal ADC reading, and the local display-toggle button.

5.1 Commands

Every supervisor-to-cabinet command shall obey the following rules:

S.1. A command frame shall begin with the character `$` and end with `#`. Numeric payloads (where present) shall be ASCII decimal digits.

S.2. A frame whose contents do not match one of the commands listed below (framing errors, incorrect length, unknown identifier, or an out-of-set numeric payload) shall be discarded silently, with no reply.

S.3. A well-formed command whose effect would not change the cabinet's state shall be ignored silently, with no reply (for example, `$CON1#` while port 1 is already connected, or `$LIM24#` while the requested limit is already `24`).

S.4. Before `$GO#` is accepted, the cabinet shall remain silent.

S.5. After `$END#` is accepted, the cabinet shall blank the display, stop transmitting, stop ADC sampling, and silently ignore every later command frame and button release.

Command	Bytes	Meaning	Accepted When
`$GO#`	4	Start simulation	Only in `WAITING`
`$END#`	5	End simulation	Only in `ACTIVE`
`$CON0#`	6	Connect port 0	In `ACTIVE`, port 0 disconnected
`$CON1#`	6	Connect port 1	In `ACTIVE`, port 1 disconnected
`$CON2#`	6	Connect port 2	In `ACTIVE`, port 2 disconnected

Command	Bytes	Meaning	Accepted When
`$DIS0#`	6	Disconnect port 0	In `ACTIVE`, port 0 connected
`$DIS1#`	6	Disconnect port 1	In `ACTIVE`, port 1 connected
`$DIS2#`	6	Disconnect port 2	In `ACTIVE`, port 2 connected
`$LIM00#`	7	Set `requested_limit` to `00`	In `ACTIVE`, requested differs
`$LIM08#`	7	Set `requested_limit` to `08`	In `ACTIVE`, requested differs
`$LIM16#`	7	Set `requested_limit` to `16`	In `ACTIVE`, requested differs
`$LIM24#`	7	Set `requested_limit` to `24`	In `ACTIVE`, requested differs

Commands are split into lifecycle commands and in-run commands. `$GO#` and `$END#` take effect as soon as their complete `$...#` frame is parsed. `$CONp#`, `$DISp#`, and `$LIMxx#` take effect only at the next 100 ms tick boundary; the full timing rules are in Section 7.

S.6. `$CONp#` and `$DISp#` shall use a single port digit `p` in `0`, `1`, `2`. Any other digit shall make the frame malformed and shall be discarded silently.

S.7. `$CONp#` shall be accepted only in `ACTIVE` when port `p` is disconnected. On acceptance, the next 100 ms tick shall mark port `p` connected and queue `$ACK01#`.

S.8. `$DISp#` shall be accepted only in `ACTIVE` when port `p` is connected. On acceptance, the next 100 ms tick shall mark port `p` disconnected and queue `$ACK02#`.

S.9. `$LIMxx#` shall use a two-digit payload `xx` equal to one of `00`, `08`, `16`, or `24`. Any other payload shall make the frame malformed and shall be discarded silently.

S.10. `$LIMxx#` shall be accepted only in `ACTIVE` when `xx` differs from the current `requested_limit`. On acceptance, the next 100 ms tick shall update the cabinet-wide `requested_limit` and queue `$ACK03#`.

S.11. Malformed, lifecycle-invalid, and idempotent[1] commands shall not change state and shall not produce an acknowledgement.

[1] A command is idempotent when applying it again would not change the cabinet state.

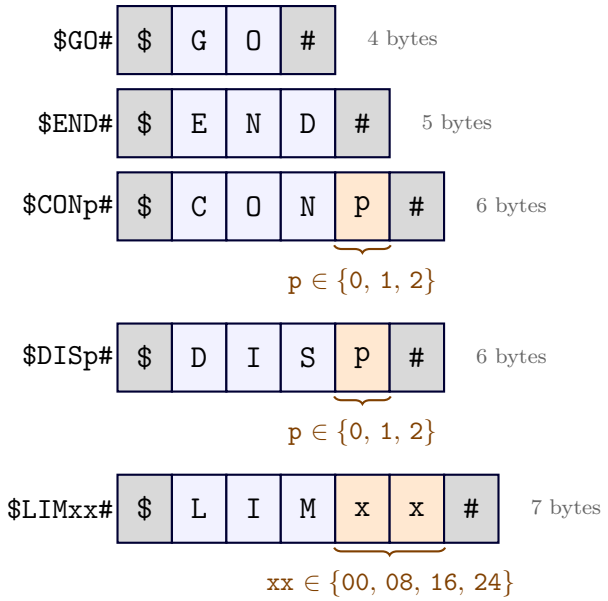

Figure 4: Supervisor command frame layouts. Orange cells are payload fields.

5.2 ADC Module

The cabinet reads a potentiometer on `AN12 / RH4` as its thermal input.

S.12. The ADC shall produce 10-bit right-adjusted output.

S.13. Conversion completion shall signal through the ADC interrupt; polling is not acceptable.

No conversion to degrees Celsius is required: every threshold in this assignment is defined directly on the raw `ADRES` value.

5.2.1 Sampling Cadence

S.14. While the cabinet is in `ACTIVE`, one conversion shall be triggered every 500 ms.

S.15. The cabinet shall trigger one conversion immediately after `$GO#` is accepted, so a valid reading is available for the cold-start rule (Section 8).

5.2.2 Thermal Classifier

The thermal classifier maps each completed ADC reading to a band by direct comparison against the threshold values listed below.

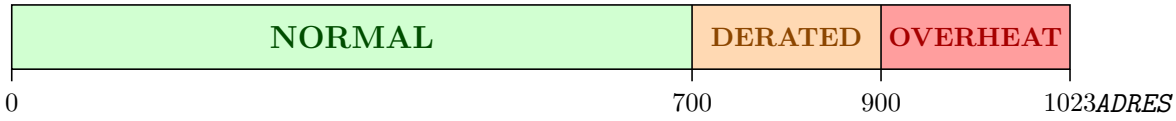

Figure 5: Thermal band classification of the `ADRES` reading.

ADRES	thermal_band	Cap
< 700	NORMAL	24
700--899	DERATED	08
>= 900	OVERHEAT	00

S.16. On every completed ADC conversion, the cabinet shall update `thermal_band` and the band cap to the values listed in the table above.

S.17. The band shall depend only on the latest `ADRES` value; band transitions in either direction use the same thresholds shown in the table.

5.2.3 Effective Current Limit

The cabinet's *effective* current limit `ee` reported in `STS` is the minimum of the supervisor's requested limit and the band's cap:

$$\texttt{ee} \;=\; \min(\texttt{requested\_limit},\ \texttt{cap})$$

S.18. `requested_limit` is set by the most recent accepted `$LIMxx#` command. Its initial value at `$GO#` is `00` (see Section 8).

S.19. The cap depends on the current `thermal_band` per Section 5.2.2.

S.20. `ee` shall be derived immediately before each `STS` frame is built as `min(requested_limit, band_cap)`.

For example, suppose `$LIM24#` has been accepted, so `requested_limit = 24`. The reported `ee` then follows the live ADC band:

Current band	Band cap	Next `STS` reports
NORMAL	24	ee = 24
DERATED	08	ee = 08
OVERHEAT	00	ee = 00

If the pot moves from `DERATED` back to `NORMAL`, the next `STS` reports `ee = 24` without another `$LIM24#`.

5.3 Operator Button

The cabinet has one operator button:

S.21. The cabinet shall read `RB6` via PORTB interrupt-on-change.

S.22. `RB6` actions shall fire only on the release (rising) edge of the button.

S.23. Each accepted `RB6` release shall advance the 7-segment display page (Section 6.2).

Other PORTB pushbuttons (`RB4`, `RB5`, `RB7`) are unused in this assignment and may be left unconnected.

6 Outputs from the Cabinet

This part covers what the cabinet transmits and displays: framed status and acknowledgement messages over EUSART, and the local 7-segment display.

6.1 Messages

The cabinet shall emit exactly two kinds of frames:

- `STS` Periodic status frame reporting the cabinet's current state.
- `ACK` Acknowledgement of an accepted, state-changing supervisor command.

The following transmission rules apply:

S.24. Every cabinet-to-supervisor frame shall begin with `$` and end with `#`.

S.25. At every 100 ms transmission slot during `ACTIVE`, the cabinet shall transmit exactly one frame: an `ACK` if one is pending, otherwise a fresh `STS`.

S.26. After transmitting an `ACK`, the cabinet shall clear the pending-`ACK` state so the next slot defaults to `STS`.

S.27. There shall be at most one `ACK` pending at any time.

The simulator used for testing and grading sends at most one in-run command between two consecutive 100 ms transmission slots, so a single pending-`ACK` slot is sufficient. During one tick, the cabinet can apply at most one pending in-run command; if that command is accepted, its `ACK` is transmitted at the end of the same tick and the slot is cleared.

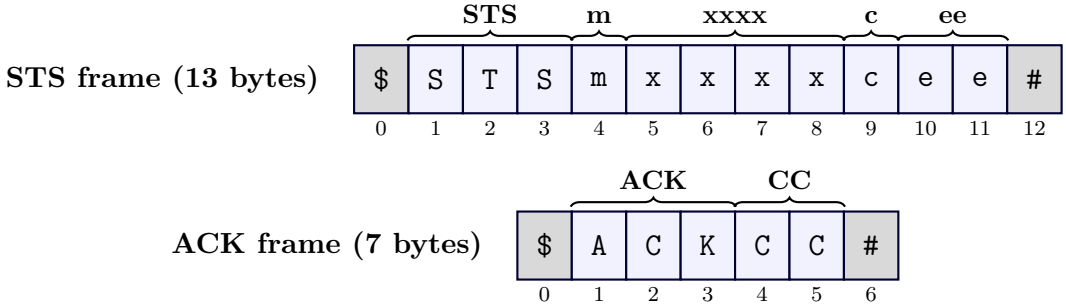

Figure 6: STS (13 bytes) and ACK (7 bytes) frame layouts.

6.1.1 Status Message

A status frame shall have the form

```
$STSmxxxxcee#
```

with an 8-byte body after the `STS` prefix. The four fields shall always appear in the order shown:

Field	Width	Values	Meaning
m	1 char	N, D, H	Current cabinet mode
xxxx	4 digits	0000–1023	Most recent raw ADRES reading
c	1 digit	0–7	Connected-port mask (bit i set ⇔ port i connected)
ee	2 digits	00, 08, 16, 24	Cabinet-wide effective current limit (in amperes)

S.28. The xxxx field shall contain the most recent completed ADC conversion result.

S.29. The c field shall be a 3-bit connected-port mask: bit 0 for port 0, bit 1 for port 1, and bit 2 for port 2. For example, c = 5 means ports 0 and 2 are connected and port 1 is disconnected.

S.30. The ee field shall be derived as min(requested_limit, thermal_cap) and shall apply cabinet-wide; it shall not be split across connected ports.

S.31. The m field shall encode the current thermal band using the table below.

m value	Band	Cap on ee
N	NORMAL	24
D	DERATED	08
H	OVERHEAT	00

6.1.2 Acknowledgement Message

An acknowledgement frame shall have the form

$ACKCC#

where CC is a 2-digit decimal code identifying which command was accepted. Four codes are defined:

Code	Acknowledges
00	$GO# accepted (cabinet has entered ACTIVE)
01	$CONp# accepted with state change (port p connected)
02	$DISp# accepted with state change (port p disconnected)
03	$LIMxx# accepted with state change (requested_limit updated)

S.32. An ACK shall be emitted only for a command that is valid in the current lifecycle and causes a state change.

S.33. $GO# shall queue ACK00 when the cabinet enters ACTIVE.

S.34. Idempotent $CONp#, $DISp#, and $LIMxx# commands shall not produce an ACK.

S.35. ACK01 and ACK02 shall not encode the port number; the next STS frame's c mask carries the updated port state.

S.36. $END# shall not produce an ACK. After $END# is accepted, the cabinet shall stop transmitting.

6.2 7-Segment Display

The display has two pages, cycled by `RB6`. Page `0` is the default after `$GO#`. Digit 3 is the leftmost position and digit 0 is the rightmost. Each layout below shows a representative sample value, and the label above each digit identifies the field displayed in that position.

- **Page** `0` the most recent raw ADC reading, as a 4-digit decimal number from `0000` to `1023`.

Figure 7: Page `0` example: ADC reading `0480`.

- **Page** `1` the effective current limit (`00`, `08`, `16`, or `24`) in the leftmost two digits, a blank in digit 1, and the connected-port mask (`0`–`7`) in digit 0.

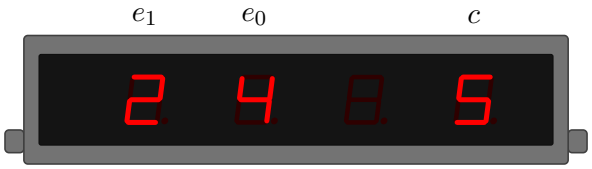

Figure 8: Page `1` example: `ee = 24`, mask `c = 5`.

S.37. During `WAITING` and `END`, the 7-segment display shall be blank.

S.38. During `ACTIVE`, the 7-segment display shall continuously show one of the two pages above.

S.39. Each accepted release of `RB6` shall toggle the page: `0` → `1` and `1` → `0`.

S.40. Page `0` shall be the active page at `$GO#` acceptance.

S.41. The display shall be multiplexed using a Timer1 interrupt at a rate fast enough to avoid visible flicker.

S.42. Page `0`'s `xxxx` field shall reflect the most recent completed ADC conversion no later than the next 100 ms tick's display-buffer refresh. Page `1`'s `ee` and `c` fields shall update at every tick.

6.2.1 Driving the Display

The display is multiplexed: `PORTJ[0:6]` sets the segment pattern and `PORTH[0:3]` selects the active digit.

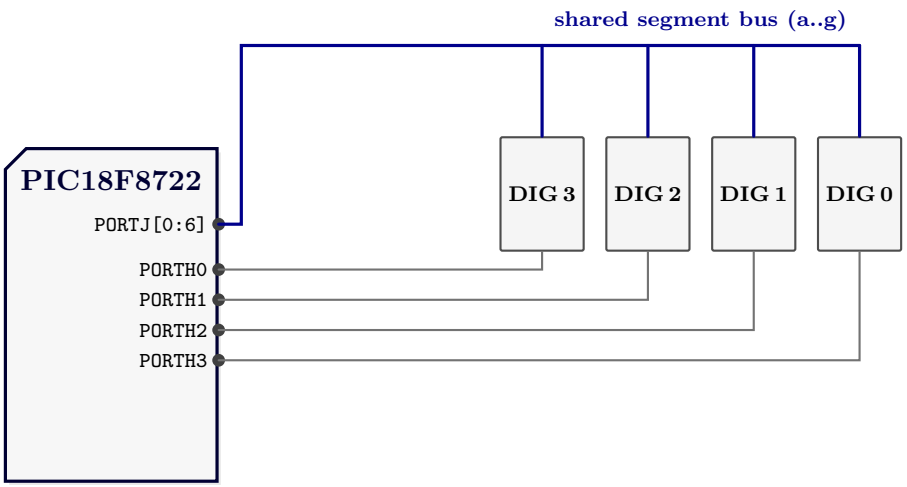

Figure 9: Multiplex wiring. `PORTJ[0:6]` is shared by all four digits; `PORTH0` through `PORTH3` select digits `3` through `0` (left to right).

S.43. `PORTJ[0:6]` shall drive segments a, b, c, d, e, f, g.

S.44. `PORTH0`, `PORTH1`, `PORTH2`, and `PORTH3` shall select digits `3`, `2`, `1`, and `0`, respectively.

S.45. The 7-segment display shall not exhibit visible flicker or digit-to-digit ghosting.

Scan order (recommended). On each Timer1 interrupt: (1) disable every digit by clearing only the `PORTH[0:3]` digit-select bits; (2) write the next segment pattern to `LATJ`; (3) enable the chosen digit by setting its `PORTH` bit. This avoids ghosting; other schemes are acceptable as long as the visible display is correct.

7 Timing Semantics

Every timing rule in this assignment is defined with respect to a common 100 ms tick grid. `$GO#` marks the zero of this grid.

S.46. The first tick shall occur 100 ms after the complete `$GO#` frame is parsed and accepted. Subsequent ticks shall occur every 100 ms.

S.47. `$GO#` and `$END#` are lifecycle commands. They shall take effect immediately after their complete `$...#` frame is parsed, not at a tick boundary.

S.48. `$CONp#`, `$DISp#`, and `$LIMxx#` are in-run commands. A parsed in-run command shall be held for the next tick, where it is evaluated against the current cabinet state.

S.49. If the pending in-run command is accepted and changes state, its `ACK` shall be transmitted at that same tick instead of `STS`. The updated state shall appear in later `STS` frames.

S.50. A completed ADC conversion shall affect the reported thermal mode and cap at the first tick whose ADC-processing step sees that completed result.

S.51. At each tick during `ACTIVE`, the cabinet shall transmit exactly one frame: the pending `ACK`, if any; otherwise a fresh `STS`.

The simulator used for testing and grading sends at most one in-run command between two consecutive ticks. Therefore, a correct cabinet implementation does not need an acknowledgement queue.

At each 100 ms tick, the cabinet shall produce externally visible behavior equivalent to Algorithm 1.

Algorithm 1: Required within-tick semantic order

```
1  Parse any completed incoming command frame, if present
2  if the cabinet is no longer in ACTIVE then
3  |   return
4  end
5  Apply the pending in-run command, if any; queue its ACK if accepted
6  Process any completed ADC result; select the reported thermal mode and cap
7  Process the RB6 release flag; select the next display page if pressed
8  Compute ee as the minimum of the requested current limit and the cap
9  Refresh the 7-segment display buffer for the active page
10 Transmit the queued ACK if any; otherwise transmit STS
```

The reason for the lifecycle check's early return is that if the parsing step accepts a complete `$END#` frame, the cabinet immediately enters `END`. The early return stops the tick handler before it can update state or transmit one more frame after `$END#`.

8 Initial Conditions

When `$GO#` is accepted, the cabinet shall enter `ACTIVE` with the following externally visible starting behavior:

S.52. No port shall be connected initially; therefore `STS.c` shall report `0` until accepted `$CONp#` commands change the reported mask.

S.53. The requested current limit shall initially be `00`.

S.54. The 7-segment display shall start on display page `0`.

S.55. The reported thermal mode shall be `NORMAL` provisionally, until the first ADC conversion result is processed.

S.56. The cabinet shall start one ADC conversion immediately after accepting `$GO#`. When it completes, the reported thermal mode and thermal cap shall be selected according to Section 5.2.2.

S.57. Because the requested current limit starts at `00`, the cabinet shall report `ee = 00` until an accepted `$LIMxx#` raises the requested limit and the thermal band permits it.

9 Implementation Requirements

The program shall satisfy the following requirements. Figure 10 shows the serial, ADC, button, and 7-segment wiring.

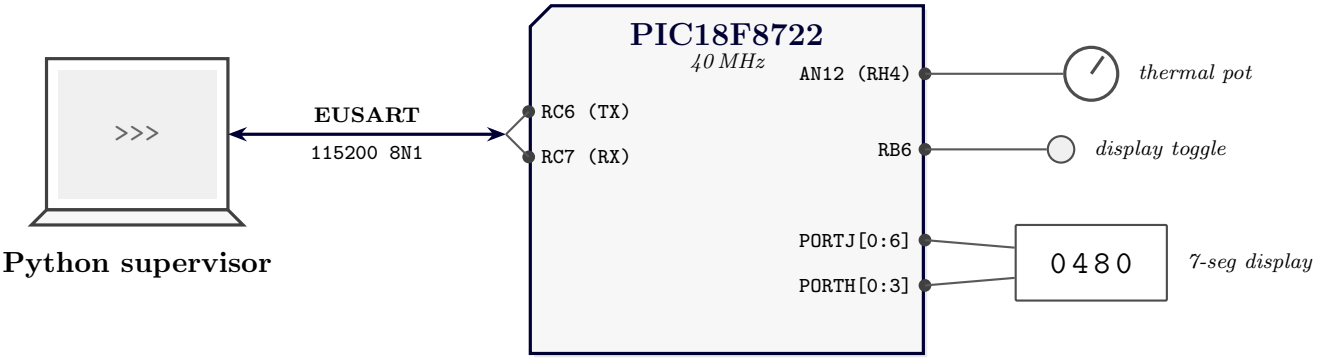

Figure 10: PIC18F8722 pin assignments.

S.58. The program shall be written in C and compiled with MPLAB XC8 v2.30. The course lab image provides MPLAB X IDE v5.45 and XC8 at `/opt/microchip/xc8/v2.30/`.

S.59. The target device shall be the PIC18F8722 clocked at 40 MHz, configured via the supplied `pragmas.h`.

S.60. EUSART communication shall use 115200 baud, 8 data bits, no parity, and 1 stop bit.

S.61. EUSART reception and transmission shall be interrupt-driven. Incoming frames shall be parsed from the receive path and outgoing frames shall be sent through a transmit buffer, not by busy-waiting in the main loop or an interrupt service routine.

S.62. The 100 ms tick and the 500 ms ADC sampling cycle shall both be driven by timer interrupts. The 500 ms cadence may be derived as a counter on the 100 ms tick.

S.63. ADC conversion completion shall be detected through the ADC interrupt; polling on `ADCON0.GO` for completion is not acceptable.

S.64. `RB6` activity shall use PORTB interrupt-on-change; release edges shall be detected from the captured PORTB value, not by polling.

S.65. EUSART receiver overrun and framing errors shall be recovered silently: the erroneous byte shall be discarded, reception shall be re-enabled as required, and no `ACK` shall be emitted. Consult the PIC18F8722 datasheet for the exact register sequence.

S.66. Received frames whose body does not match any supervisor command, or whose framing is malformed, shall be discarded with no reply.

S.67. The UART input and output ring buffers shall each hold at least 32 bytes.

S.68. Interrupt service routines shall not invoke `printf`, shall not busy-wait, and shall not perform large string formatting. Non-trivial work shall be deferred to the main loop or to the periodic tick handler.

S.69. The source code shall carry brief explanatory comments wherever the design intent is not evident from identifier names alone.

10 Grading and Hand-In

10.1 Rubric

The assignment is graded out of 100 points. Each row below is graded independently, so partial solutions keep the points for behaviors they implement correctly.

Category	Item	Pts.
Serial protocol (30)	Recognize `$...#` frames; ignore bytes outside frames	4
	Discard malformed, wrong-length, and invalid-payload frames silently	3
	Accept `$GO#` only in `WAITING`; accept `$END#` only in `ACTIVE`	6
	Parse port digit `p`; `$CONp#` sets and `$DISp#` clears the matching `c` bit	6
	Parse `$LIMxx#`; update `requested_limit` for `00/08/16/24`	4
	Emit correct `ACK/STS`; one cabinet frame per active tick	7
ADC & thermal (25)	Configure ADC for `AN12` with correct clock and format	5
	ADC conversion triggered every 500 ms	5
	Handle ADC completion by interrupt, not busy-wait polling	5
	Apply thermal thresholds: `<700`, `700-899`, `>=900`	5
	Report latest `ADRES` as `xxxx`; report band as `m`	5
Current limit (15)	Initialize `requested_limit = 00`; update only by accepted `$LIMxx#`	5
	Derive `thermal_cap` from band: `24/08/00`	4
	Recompute `ee = min(requested_limit, thermal_cap)` every tick	4
	Report `ee` as one cabinet-wide value	2
Timing (15)	Generate the 100 ms cabinet tick	5
	Apply in-run commands at the next tick	4
	Transmit exactly one cabinet frame per active tick	3
	After `$END#`: transmit nothing and blank the display	3
Display & RB6 (15)	Multiplex all four 7-segment digits using Timer1	5
	Display blank in `WAITING` and after `$END#`	2
	Page 0 shows 4-digit decimal `ADRES` with leading zeros	3
	Page 1 shows `ee`, blank digit, and `c` mask	3
	Toggle display page on `RB6` release edge only	2
Total		**100**

10.2 Submission

Students shall submit a single archive named `the3.zip` through **ODTUClass**. The archive shall include all source files, header files, project files, and any required build configuration files. It shall also include a plain-text file named `names.txt` listing each group member's student ID, name, surname, and group number.

10.3 Academic Integrity

Students and groups may discuss concepts among themselves or with the instructor and teaching assistants. However, when it comes to the actual work, it must be done by the student or group alone. Every student in a group is expected to make a meaningful contribution to the assignment. As soon as you begin writing your solution, you must work alone. Copying text directly from someone else (whether by copying files, typing from someone else's notes, or typing while they dictate) constitutes plagiarism. This is true regardless of the source: classmate, former student, website, program listing, or otherwise. Plagiarism, even on a small part of the program, is cheating. Starting from code that you did not write and modifying it to look like your own is also cheating. Aiding another student's cheating constitutes cheating. Leaving your program in plain sight or leaving a computer without logging out (thereby leaving your programs open to copying) may constitute cheating depending on the circumstances. Automated tools are used to detect plagiarism. Both parties involved in any incident of cheating will be subject to disciplinary action.

Using Large Language Models (LLMs) and other AI tools. AI can be a useful learning resource, but using it to generate code that you are expected to write yourself is considered cheating. Using code from any other resource (whether generated by humans or AI) is likewise considered cheating.